

Введение

Рост объема научных материалов усложняет поиск и использование релевантной информации. Обычный поиск по ключевым словам плохо работает в случаях, когда одна и та же идея выражается разными формулировками, а ответы на содержательные вопросы требуют не одного фрагмента, а набора связанных фрагментов из корпуса. При этом универсальные языковые модели без опоры на локальную базу знаний не гарантируют привязку ответа к конкретным документам пользователя. Поэтому актуальной является задача разработки автоматической справочной системы, которая отвечает в контексте выбранного научного направления и опирается на загруженный корпус материалов.

Целью курсового проекта является разработка автоматической справочной системы на основе языковых моделей, обеспечивающей ответы по документам выбранного научного направления с использованием комбинированного retrieval-подхода. В качестве основы решения используется двухэтапный поиск: сначала выполняется отбор кандидатов по векторному сходству, затем результаты уточняются с учетом графа сущностей и их связей внутри корпуса.

Для достижения поставленной цели необходимо решить следующие задачи:

1. проанализировать особенности автоматических справочных систем для научных материалов и определить требования к решению;
2. спроектировать архитектуру системы, разделив исследовательский, библиотечный и прикладной уровни;
3. реализовать общее ядро пайплайнов обработки документов и поиска по корпусу в виде отдельной библиотеки;
4. реализовать исследовательский сценарий в Jupyter Notebook для демонстрации и проверки retrieval-пайплайна;

5. разработать прикладной сервис с пользовательскими воркспейсами, документами, чатами и изоляцией контекста;
6. описать полученное решение и оценить его применимость для построения научных справочных систем.

Объектом исследования являются автоматические справочные системы, использующие языковые модели для работы с тематическим корпусом документов. Предмет исследования — методы построения retrieval-слоя такой системы, включая семантический поиск, детерминированное извлечение сущностей, графовое представление корпуса и интеграцию этих механизмов в прикладной сервис.

Практическая значимость работы состоит в том, что результатом является не только демонстрационный ноутбук, но и самодостаточное программное решение из трех взаимосвязанных частей. Первая часть представляет собой Jupyter Notebook для исследования и демонстрации retrieval-пайплайна на отдельном датасете. Вторая часть оформлена как библиотека `sciguide_pipeline`, которая инкапсулирует нарезку текста, индексацию корпуса, векторный поиск и графовое перерангирование и может использоваться повторно. Третья часть реализована как сервис SciGuide, предоставляющий пользователю интерфейс работы с воркспейсами, документами и чатами и использующий библиотеку как вычислительное ядро.

Таким образом, в работе рассматривается не только локальный эксперимент с языковой моделью, но и полный путь от retrieval-логики до прикладной системы, пригодной для использования в изолированном пользовательском контексте.

Основная часть

1. Аналитический раздел

1.1. Постановка задачи разработки автоматической справочной системы

Автоматическая справочная система для научных материалов должна решать задачу содержательного ответа на пользовательский вопрос по ограниченному тематическому корпусу документов. В отличие от универсального чат-бота, такая система не должна опираться на произвольные знания модели, а обязана формировать ответ только в рамках загруженных источников. Для курсового проекта это особенно важно, поскольку целевой сценарий предполагает работу с собственным набором документов внутри выбранного научного направления.

Проблема заключается в том, что прямой поиск по ключевым словам плохо переносит вариативность научного языка. Один и тот же смысл может быть выражен разными терминами и синтаксическими конструкциями, а важные для ответа фрагменты нередко связаны не буквальным совпадением слов, а общим контекстом, совместным употреблением понятий и их местом в структуре корпуса. По этой причине пользователю недостаточно просто найти документ; системе требуется отобрать и связать несколько релевантных фрагментов, на основе которых затем формируется ответ.

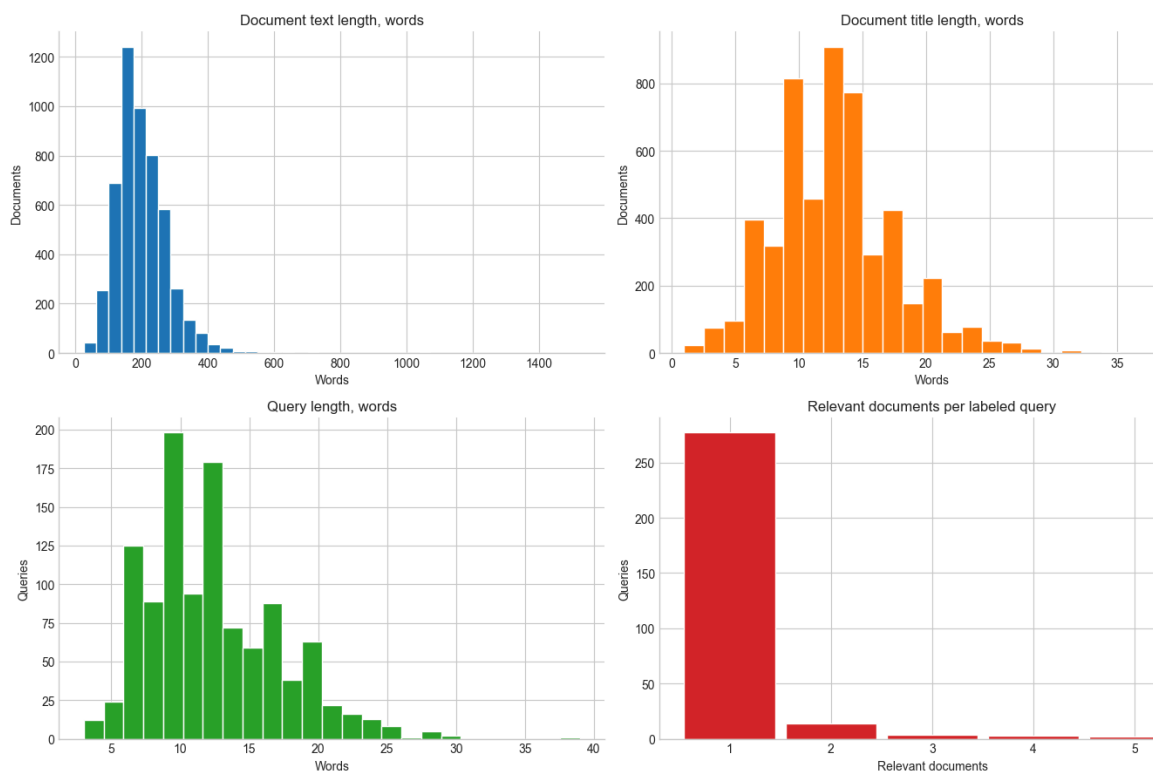
Из этого следует постановка задачи: необходимо разработать систему, которая принимает корпус документов, индексирует его, находит релевантные фрагменты по пользовательскому запросу и использует найденный контекст для генерации ответа. При этом решение должно обеспечивать изоляцию пользовательских данных, повторное использование retrieval-ядра в разных сценариях и возможность проверки качества на отдельном исследовательском датасете.

В рамках проекта задача решается на трех уровнях. Исследовательский уровень представлен Jupyter Notebook, в котором удобно проводить эксперименты и замеры. Библиотечный уровень оформлен как `sciguide_pipeline` и содержит общее ядро нарезки, индексации и поиска. Прикладной уровень представлен сервисом SciGuide, который обеспечивает работу пользователей с воркспейсами, документами и чатами. Такой разрез

позволяет не смешивать экспериментальную логику, переиспользуемое ядро и пользовательский интерфейс в одном монолитном решении.

1.2. Особенности работы с научным корпусом документов

Особенности научного корпуса хорошо видны на примере датасета SciFact, который используется в работе как основной набор данных для проверки retrieval-сценария. В ноутбуке `scifact_dataset_eda.ipynb` были получены базовые характеристики корпуса. Датасет содержит 5183 документа, 1109 запросов и 300 запросов с тестовой разметкой `qrels`; общее число размеченных пар запрос-документ равно 339. Средняя длина текста документа составляет 201,81 слова, медианная — 192 слова, а средняя длина запроса — 12,38 слова. Для размеченных запросов медианное число релевантных документов равно 1, а максимальное — 5.



Приведенные значения показывают несколько важных свойств научного корпуса.

Во-первых, запросы короткие и информационно сжатые, тогда как документы существенно длиннее и содержат больше терминов, уточнений и контекстных связей. Это означает, что поиск по точным вхождениям слов

легко теряет полезные фрагменты, если формулировка в документе не совпадает с формулировкой запроса.

Во-вторых, разметка релевантности разрежена. Для большинства размеченных запросов имеется один релевантный документ, поэтому retrieval-слой должен работать аккуратно уже на ранних этапах отбора кандидатов. Если релевантный фрагмент не попадет в первичную выборку, генерация ответа не сможет опереться на правильный контекст.

В-третьих, научные тексты характеризуются плотностью понятийных связей. Релевантность часто определяется не только тем, насколько текст похож на запрос по смыслу, но и тем, какие сущности встречаются рядом, как они связаны между собой и какие локальные кластеры образуют в корпусе. Поэтому одного keyword search недостаточно, а чистый семантический поиск по эмбедингам, хотя и улучшает полноту, не всегда различает действительно сильный контекст и просто тематически похожий фрагмент.

По этой причине в работе выбран двухэтапный retrieval-подход. На первом этапе векторный поиск обеспечивает recall и быстро находит кандидатов по семантическому сходству. На втором этапе графовое перерангирование уточняет результат, используя связи между сущностями, извлеченными из корпуса. Такой подход соответствует задаче научной справочной системы: граф не заменяет semantic search, а добавляет structural signal, позволяющий повысить качество итогового контекста для генерации ответа.

1.3. Функциональные требования к системе

Функциональные требования определяют систему не как единичный эксперимент с запросом к модели, а как многопользовательский сервис с изолированными рабочими контекстами.

Базовой сущностью системы является воркспейс. Для каждого воркспейса хранится собственный набор документов и собственный системный промпт, который задает правила поведения модели в рамках данного контекста. Воркспейсы бывают приватными и общими. Приватный воркспейс доступен только владельцу, а общий — назначенным участникам или всем пользователям системы в зависимости от настроек доступа.

Внутри воркспейса различаются две роли: администратор и пользователь. Администратор управляет составом участников, может изменять системный промпт и работать с документами воркспейса. Пользователь взаимодействует с чатами и получает ответы системы, но не изменяет состав корпуса и настройки контекста. Такое разделение необходимо, чтобы разграничить управление базой знаний и ее использование.

Документы воркспейса являются контекстной базой знаний. Система должна обеспечивать их изоляцию между разными воркспейсами, а также запускать обработку и индексацию после загрузки. Это требование напрямую влияет на архитектуру: возникает необходимость в отдельном пайплайне нарезки, извлечения сущностей, построения графа и размещения данных в хранилищах поиска.

Чаты являются личными сущностями пользователя внутри воркспейса. Каждый пользователь может создавать неограниченное число чатов, при этом история сообщений должна сохраняться и быть доступной только владельцу. При генерации ответа система обязана использовать три источника контекста: системный промпт воркспейса, документы текущего воркспейса и историю текущего чата. Использование данных из других воркспейсов или чужих чатов не допускается.

Таким образом, функциональные требования задают сразу несколько обязательных свойств решения: изоляцию данных, разграничение ролей, поддержку фоновой индексации, хранение истории диалога и интеграцию retrieval-слоя в пользовательский сценарий общения с системой.

1.4. Выводы по аналитическому разделу

Проведенный анализ показывает, что разрабатываемая система должна решать две взаимосвязанные, но разные задачи. Первая задача — качественный retrieval по научному корпусу, где важны полнота первичного поиска и учет структурных связей между понятиями. Вторая задача — корректная организация пользовательского сервиса с воркспейсами, ролями, документами, чатами и изоляцией данных.

Из этого следуют архитектурные требования к решению.

1. Retrieval-ядро должно быть выделено в самостоятельный переиспользуемый модуль, чтобы одинаково работать и в исследовательском ноутбуке, и в прикладном сервисе.
2. Индексация документов должна быть отделена от пользовательского запроса, поскольку обработка корпуса требует отдельного конвейера и фонового выполнения.
3. Хранилища прикладных сущностей и хранилища поиска должны быть разведены по назначению: пользовательские данные целесообразно хранить в транзакционном контуре, а векторные и графовые представления — в специализированных системах.
4. Архитектура backend-части должна сохранять разделение ответственности между слоями предметной области, сценариев использования, инфраструктуры и представления, что соответствует выбранным архитектурным гайдлайнам проекта.

С учетом этих требований обоснованным является следующее решение: построить систему SciGuide как сервис на базе clean architecture, выделить библиотеку `sciguide_pipeline` в качестве общего вычислительного ядра и использовать двухэтапный retrieval, объединяющий векторный поиск и графовое перерангирование. Такой подход позволяет совместить исследовательскую воспроизводимость, переиспользуемость кода и прикладную пригодность решения.

2. Проектирование и реализация программной системы

2.1. Архитектурные требования к системе

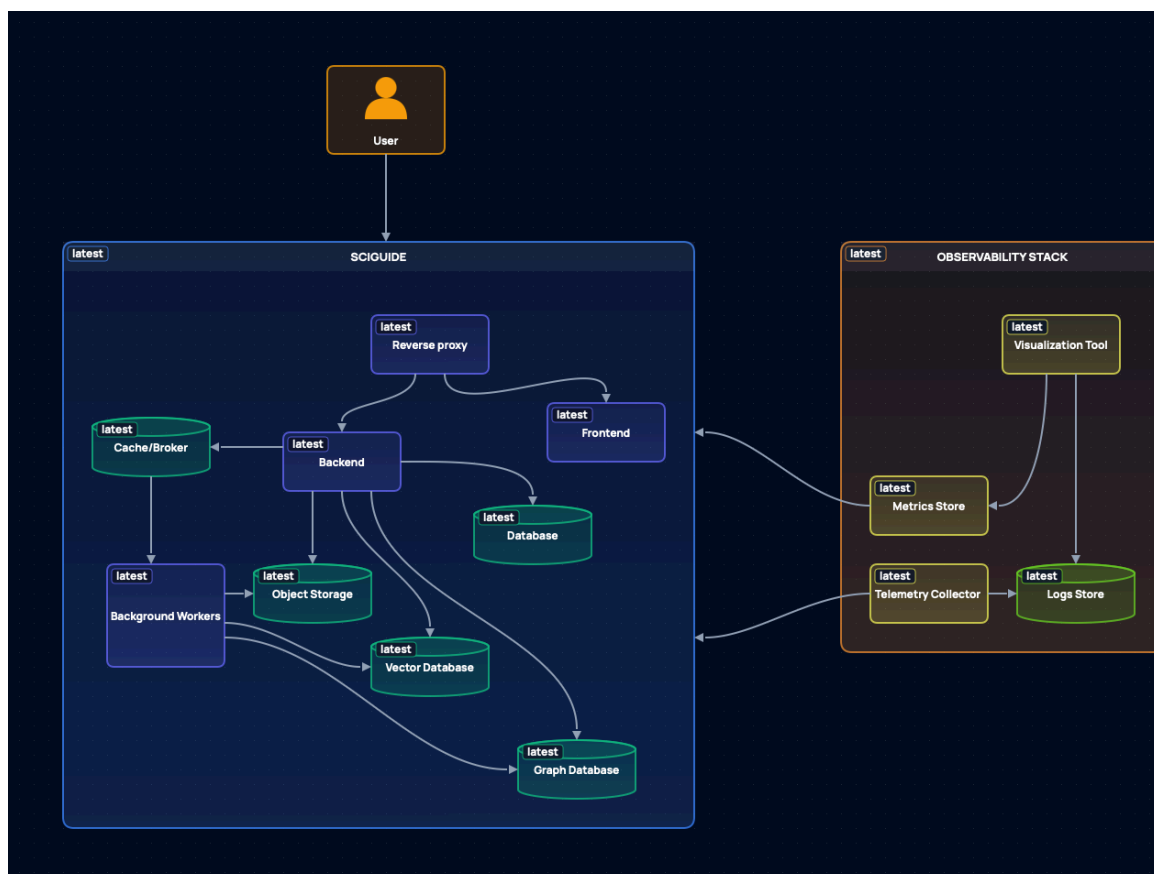
Требования к программной системе напрямую следуют из постановки задачи и функциональных требований. Во-первых, система должна изолировать данные разных воркспейсов: документы, системный промпт, чаты и история сообщений не должны смешиваться между пользователями и рабочими контекстами. Во-вторых, требуется разделить прикладные данные и данные retrieval-контура. Пользовательские сущности удобно хранить в транзакционном хранилище, тогда как векторные представления и граф связей должны размещаться в специализированных системах.

Отдельным требованием является фоновая индексация документов. Загрузка файла не должна блокировать пользовательский интерфейс, поэтому обработка корпуса вынесена в асинхронный контур. Кроме того, retrieval-ядро должно использоваться в двух режимах: в исследовательском ноутбуке и в прикладном сервисе. Это привело к выделению общей библиотеки `sciguide_pipeline`.

Архитектурные гайдлайны проекта задают использование clean architecture в backend-части. Это означает разделение на доменный слой, слой сценариев использования, инфраструктурный слой и слой представления. Такое разбиение уменьшает связность между бизнес-логикой и внешними технологиями и упрощает интеграцию retrieval-библиотеки в разные сценарии.

2.2. Общая архитектура системы SciGuide

На верхнем уровне система состоит из web-клиента, backend-сервиса, фонового worker-процесса, общей библиотеки пайплайнов и набора специализированных хранилищ. Общая структура показана на C4-диаграмме.



Frontend реализует пользовательский интерфейс для работы с воркспейсами, документами и чатами. **Backend** на FastAPI предоставляет HTTP API, управляет прикладными сущностями и координирует сценарии загрузки документов и генерации ответов. **Celery**-worker выполняет фоновую индексацию документов. Библиотека **sciguide_pipeline** инкапсулирует retrieval-логику и используется как worker-ом, так и backend-частью при поиске контекста для ответа.

Хранилища распределены по функциям. **PostgreSQL** используется для пользователей, воркспейсов, чатов и сообщений. **Redis** служит брокером задач и кэшем. **MinIO** хранит загруженные файлы. **Qdrant** хранит векторные представления фрагментов текста. **Neo4j** содержит граф сущностей и их связей. На входе в систему размещен **Nginx**, который выступает единой точкой доступа.

Поток обработки имеет следующий вид: документ загружается в воркспейс и сохраняется в **MinIO**; backend ставит задачу на индексацию; worker запускает библиотеку **sciguide_pipeline**; результаты нарезки и эмбединги сохраняются в **Qdrant**, а граф сущностей — в **Neo4j**; при

отправке сообщения backend выполняет retrieval по текущему воркспейсу и передает найденный контекст языковой модели.

2.3. Архитектура и реализация библиотеки `sciguide_pipeline`

Библиотека `sciguide_pipeline` выделена в отдельный технический компонент, чтобы не дублировать retrieval-логику в ноутбуке и в сервисе. Пользователь библиотеки работает с одной точкой входа — `PipelineManager`, который предоставляет доступ к двум фасадам: `chunking` и `search`.

```
with PipelineManager(...) as manager:
    manager.chunking.run([source_document])
    report = manager.search.run(query="Что такое graph-guided
retrieval?")
```

Внутри `PipelineManager` создаются и связываются основные компоненты пайплайна: `LangChainTextChunker`, `DeterministicEntityExtractor`, сервис эмбедингов, репозиторий векторного поиска `QdrantVectorRepository`, графовый репозиторий `Neo4jGraphRepository` и `WeightedScoreCombiner`. Таким образом, библиотека скрывает технические детали подключения к внешним системам, управляет клиентами, чтобы не перегружать исходящий трафик и предоставляет единый пользовательский контракт.

С технической точки зрения библиотека организована как последовательность согласованных шагов обработки, скрытых за единым API. Она задает собственные контракты для входных данных, результатов пайплайнов и точек расширения. Благодаря этому и в ноутбуке, и в backend-сервисе на вход передаются одинаковые сущности — содержимое документа и связанные метаданные, — а не объекты, специфичные для конкретного интерфейса загрузки или web-фреймворка. Такое решение делает библиотеку самостоятельным техническим модулем, который можно использовать независимо от прикладной оболочки.

2.4. Реализация прикладного сервиса

Backend реализован в соответствии с принципами clean architecture и разделен на доменные области: `auth`, `workspaces`, `workspace_documents`, `workspace_prompt`, `chats`, `messages`, `workspace_members`. Каждая область содержит собственные слои `domain`, `application`, `infrastructure` и `presentation`. Компоновка HTTP API выполняется на уровне общего роутера:

```
api_router = APIRouter(prefix="/api/v1")
api_router.include_router(auth_router)
api_router.include_router(workspaces_router)
api_router.include_router(documents_router)
api_router.include_router(chats_router)
api_router.include_router(messages_router)
```

Индексация документа реализована как отдельный сценарий использования. После загрузки файл читается из хранилища, для него обновляется статус обработки, затем вызывается индексатор на основе `PipelineManager`, а по завершении документ переводится в состояние `PROCESSED`.

```
await self._document_indexer.index(
    document=document,
    content_bytes=content_bytes,
)
```

Генерация ответа строится аналогично: сервис сначала извлекает контекст текущего воркспейса через `sciguide_pipeline`, а затем передает найденные фрагменты языковой модели. В `OpenRouterAssistantResponder` `retrieval` выполняется для коллекции, связанной с конкретным `workspace_id`, что обеспечивает изоляцию поискового контекста между воркспейсами.

Frontend реализован на Vue и выступает пользовательской оболочкой над backend API. Его задача — предоставить сценарии регистрации, работы с воркспейсами, загрузки документов, просмотра истории обработки и общения с системой в чате.

2.5. Инфраструктура развертывания

Система рассчитана на контейнерное развертывание. Основные сервисы поднимаются через Docker Compose, что позволяет воспроизвести рабочее окружение целиком: backend, frontend, worker, базы данных и retrieval-хранилища. На входе используется **Nginx**, а фоновая индексация реализована через **Celery** с **Redis** в роли брокера и backend-хранилища задач.

```
celery_app = Celery(  
    "sciguide",  
    broker=_build_redis_url(),  
    backend=_build_redis_url(),  
    include=["workspace_documents.infrastructure.tasks"],  
)
```

Backend поднимает подключения к **PostgreSQL** и **Redis** в **lifespan**-контексте FastAPI, а также публикует healthcheck и HTTP-метрики. Для сопровождения системы предусмотрены **Prometheus**, **Grafana**, **Loki**, **Grafana Alloy** и формат телеметрии **OpenTelemetry**.

В результате реализованная инфраструктура поддерживает как исследовательский режим, так и эксплуатационный сценарий: пользователь может загрузить документ, дождаться фоновой индексации и получить ответ, сформированный на основе контекста текущего воркспейса.

3. Разработка и исследование retrieval-подхода

3.1. Формализация retrieval-задачи для научного корпуса

Зафиксировать постановку задачи поиска релевантных фрагментов по пользовательскому запросу и определить, какой результат retrieval-пайплайна считается достаточным для построения ответа.

3.2. Сравнительный анализ подходов к поиску по научным документам

Сопоставить поиск по ключевым словам, чистый semantic search, использование LLM без retrieval и гибридный retrieval с дополнительным структурным сигналом. Показать сильные и слабые стороны каждого подхода и обосновать выбор комбинированного решения.

3.3. Модель выбранного retrieval-пайплайна

Описать концептуальную схему пайплайна: разбиение документов на фрагменты, получение векторных представлений, извлечение сущностей, построение графа связей и перерангирование кандидатов.

3.4. Конфигурация и шаги исследуемого пайплайна

Показать, какие этапы включены в итоговый пайплайн, какие варианты обработки и поиска рассматривались, и чем выбранная конфигурация отличается от более простых альтернатив.

3.5. Методика эксперимента и метрики оценки

Описать используемый датасет SciFact, подготовку корпуса и запросов, исследовательский сценарий в Jupyter Notebook, а также метрики оценки retrieval-качества.

3.6. Анализ результатов исследования

Привести результаты проверки пайплайна, интерпретировать полученные значения метрик и сформулировать выводы о применимости выбранного подхода для научной справочной системы.